Introduction to Object-oriented programming and software development

Lecture 1 – Part 1

Based on Slides of Dr. Norazah Yusof

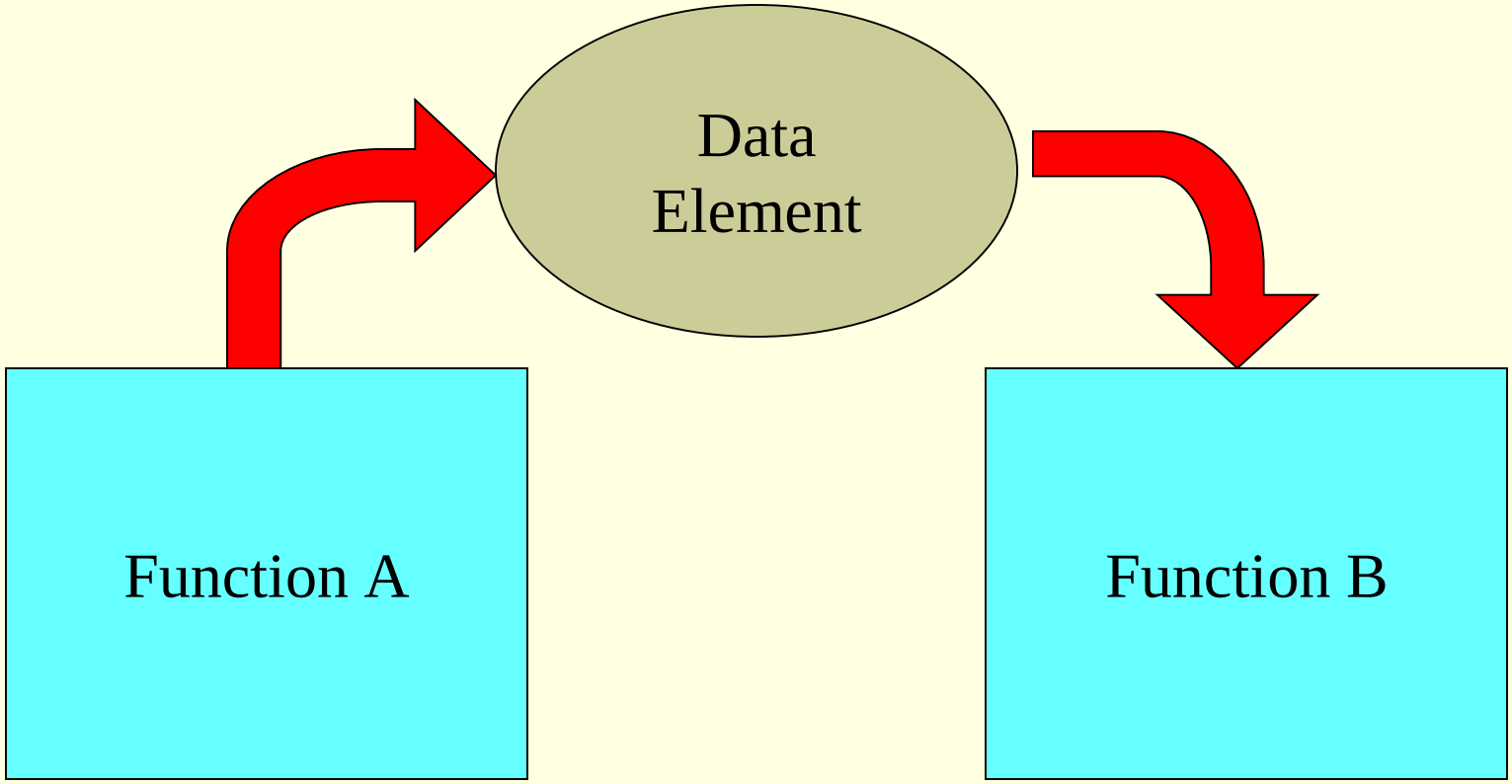
Procedural Programming

- Traditional programming languages were procedural.
  - C, Pascal, BASIC, Ada and COBOL
- Programming in procedural languages involves choosing data structures (appropriate ways to store data), designing algorithms, and translating algorithm into code.

Procedural Programming

- In procedural programming, data and operations on the data are separated.
- This methodology requires sending data to procedure/functions.

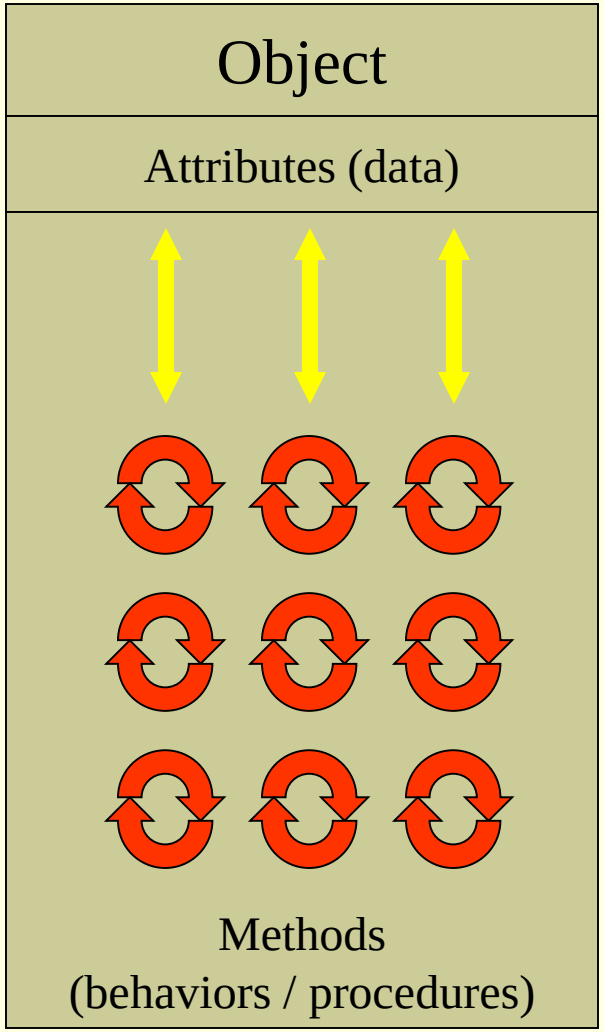
Procedural Programming



Object-Oriented Programming

- Object-oriented programming (OOP) is centered on creating **objects** rather than procedures/ functions.
- Objects are a melding of **data** and **procedures** that manipulate that data.
- Data in an object are known as ***attributes***.
- Procedures/functions in an object are known as ***methods***.

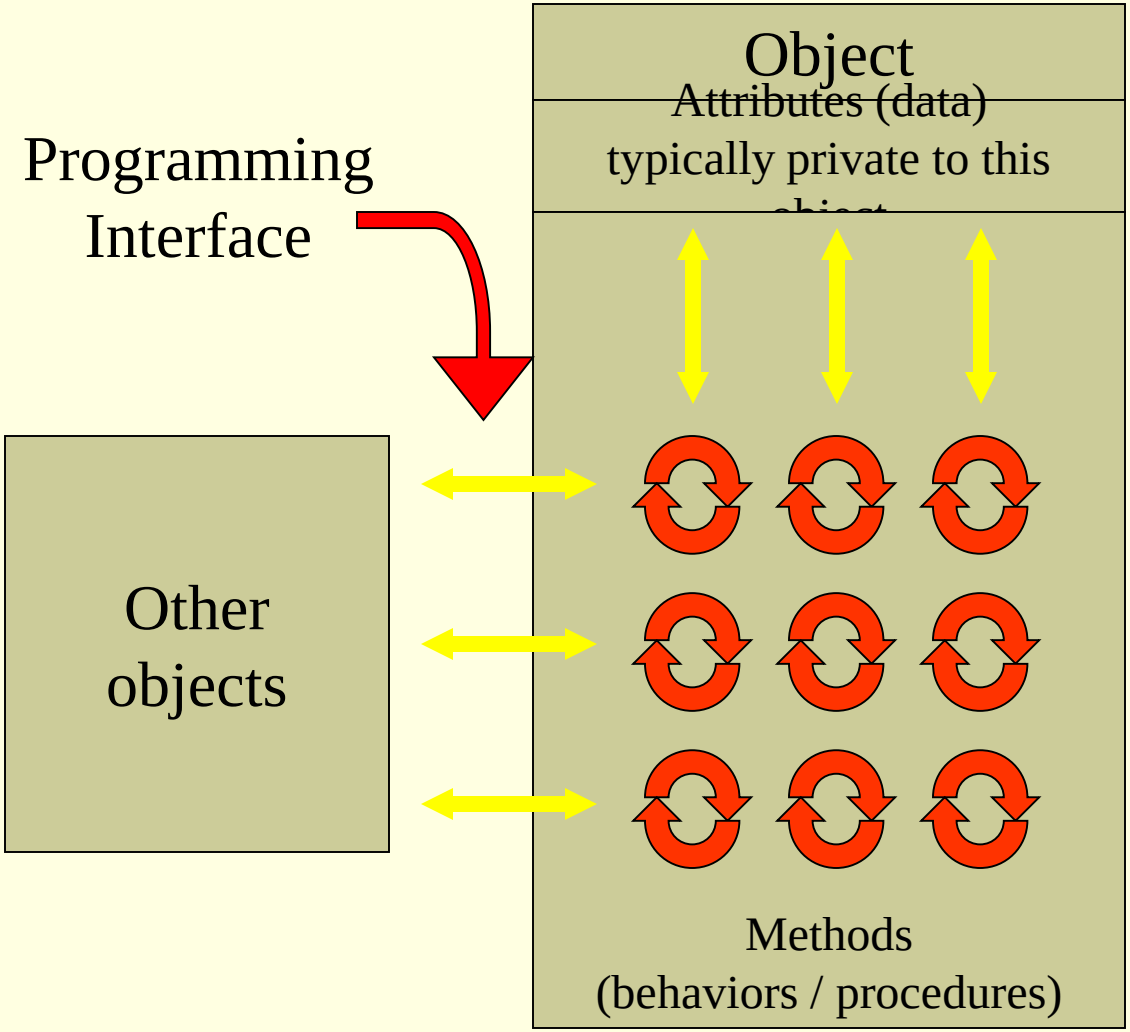
Object-Oriented Programming



Object-Oriented Programming

- Object-oriented programming combines data and behavior (or method) via ***encapsulation***.
- ***Data hiding*** is the ability of an object to hide data from other objects in the program.
- Only an objects **methods** should be able to directly manipulate its **attributes**.
- Other objects are allowed to manipulate an object's attributes via the object's methods.
- This indirect access is known as a ***programming interface***.

Object-Oriented Programming



Object-Oriented Programming Languages

- Pure OO Languages

  Smalltalk, Eiffel, Actor, Java

- Hybrid OO Languages

  C++, Objective-C, Object-Pascal

The Vocabulary of OOP - Class

- The most important term is the **class**.
  - A class is the template or blueprint from which objects are actually made.
  - All codes that you write in Java is inside a class.
  - The standard Java library supplies several thousand classes.
  - You can create your own classes in Java to describe the objects of the problem domains of your applications and to adapt the classes that are supplied by the standard library to your own purposes.
    - A class encapsulates the attributes and actions that characterizes a certain type of object.

The Vocabulary of OOP - Objects

- Classes can be used to **instantiate** as many **objects** as are needed.
- Each object that is created from a class is called an ***instance*** of the class.
- A program is simply a collection of objects that interact with each other to accomplish a goal.

Classes and Objects

The *Vehicle* class defines the attributes and methods that will exist in all objects that are an instances of the vehicle class.

Car class

Kancil object

The Kancil object is an instance of the Car class.

The Nazaria object is an instance of the Car class.

Nazaria object

The Vocabulary of OOP - Encapsulation

- **Encapsulation** is a key concept in working with objects.
  - Combining data and behavior in one package and hiding the implementation of the data from the user of the object.
  - The data in an object are called its **instance fields**, and the procedures that operate on the data are called its **methods**.
  - The key to making encapsulation work is to have **methods never directly access instance fields** in a class other than their own.
  - Programs should interact with object data only through the object's methods.
  - By encapsulating object data, you maximize reusability, reduce data dependency, and minimize debugging time.

The Vocabulary of OOP – Data Hiding

- Data hiding is important for several reasons.
  - It protects of attributes from accidental corruption by outside objects.
  - It hides the details of how an object works, so the programmer can concentrate on using it.
  - It allows the maintainer of the object to have the ability to modify the internal functioning of the object without “breaking” someone else's code.

The Vocabulary of OOP – Code Reusability

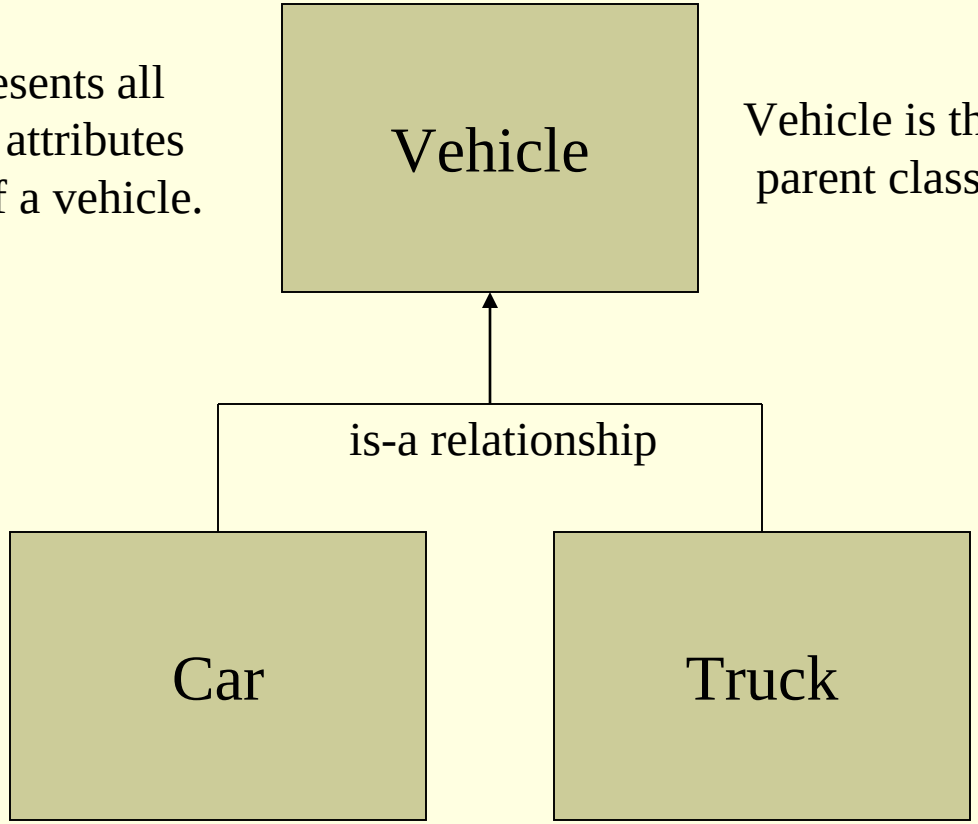
- Object-Oriented Programming (OOP) has encouraged component reusability.
- A component is a software object contains data and methods that represents a specific concept or service.
- Components typically are not stand-alone programs.
- Components can be used by programs that need the component's service.
- Reuse of code promotes the rapid development of larger software projects.

The Vocabulary of OOP – Inheritance

- *Inheritance* is the ability of one class to extend the capabilities of another.
  - it allows code defined in one class to be reused in other classes
- Consider the class Car. A Car is a specialized form of the Vehicle class.
  - So, is said that the Vehicle class is the **base** or **parent** class of the Car class.
  - The Car class is the **derived** or **child** class of the Vehicle class.

The Vocabulary of OOP - Inheritance

Vehicle represents all of the generic attributes and methods of a vehicle.



Vehicle is the parent class.

Car and Truck are child classes of Vehicle.

Car and Truck are Specialized versions of a Vehicle.

The Vocabulary of OOP – Polymorphism

- *Polymorphism* means "many forms."
  - A reference variable is always of a single, unchangeable type, but it can refer to a subtype object.
  - A single object can be referred to by reference variables of many different types—as long as they are the same type or a supertype of the object. The reference variable's type (not the object's type), determines which methods can be called!
- Polymorphism allows extensibility.

Self-test: Introduction to Object Oriented Programming

1. State the differences between procedural programming and Object Oriented Programming.
2. What is an Object and what is a Class? What is the difference between them?
3. What is an Attribute?
4. What is a Method?
5. What is encapsulation? How it relates to information hiding?
6. What is inheritance? How it relates to polymorphism?